

5 - FEM Implementation

For the python implementation, we used:

- The global and elemental indexing schemes for the nodes and elements.
- The previously derived formulas of the coefficients for the basis functions ϕ_i , elementary stiffness elements A_{ij}^k , and elementary load elements F_i^k .
- Initial and Boundary condition implementation by:
 - Zeroing out the rows and columns of the stiffness matrix corresponding to the boundary nodes.
 - Setting the diagonal entries of the stiffness matrix for the boundary nodes to 1.
 - Setting the corresponding entries in the load vector to the boundary condition values.
- After assembly, we used Scipy's linear solver to generate the solution vector.

The general structure of the code is as follows:

1. Define mesh dimensions and parameters.
2. Generate a global node index, element connectivity, and elemental nodal value matrices.
3. Generate elemental stiffness matrices and load vectors, and assemble them into the global stiffness matrix and load vector.
4. Enforce initial and boundary conditions by modifying the stiffness matrix and load vector.
5. Solve the resulting linear system to obtain the solution at the nodes.
6. Plotting and error computation.

Listing 1: FEM Python Implementation

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.linalg import solve
4
5  # Space discretization
6  N=6
7  dx=1/N
8  X_NODES = np.linspace(0, 1, N+1)
9  # Time discretization
10 M=4
11 dt = 1/M
12 T_NODES = np.linspace(0, 1, M+1)
13 # Triangular Element area
14 AREA = (dx*dt)/2
15
16 def u_exact(x,t):
17     return np.sin(np.pi*x)*np.cos(t)
18
19 def f(x,t):
20     return (np.pi**2 - 1)*np.sin(np.pi*x)*np.cos(t)
21
22 def generate_global_index_matrix():
23     '''
24     Generates a matrix of global indices for the nodes in the mesh as helper
25     for generate_connectivity_matrix_and_nodal_values method.
26     '''
27     global_index_matrix = np.zeros((M+1, N+1), dtype=int)
28     count = 0
29     for i in range(M, -1, -1):
30         for j in range(N+1):
31             global_index_matrix[i, j] = count
32             count += 1
33     return global_index_matrix
34
35 def generate_connectivity_matrix_and_nodal_values(global_index_matrix):
36     '''
37     Generates the connectivity matrix and nodal values for the elements in the
38     mesh.
39
40     The nodal values have the format [[x1, t1], [x2, t2], [x3, t3]] for each
41     element, where (x1, t1), (x2, t2), and (x3, t3) are the coordinates of the
42     three nodes of the triangle. The connectivity matrix has the format [[n1, n2
43     , n3], ...] where n1, n2, and n3 are the global indices of the nodes in the
44     triangle.
45     '''

```

```

41
42     connectivity_matrix = []
43     nodal_values=[]
44     for i in range(M, 0, -1):
45         for j in range(N):
46             bottom_left = global_index_matrix[i, j]
47             bottom_right = global_index_matrix[i, j+1]
48             top_right = global_index_matrix[i-1, j+1]
49             top_left = global_index_matrix[i-1, j]
50             connectivity_matrix.append([bottom_left, bottom_right, top_right])
51             connectivity_matrix.append([bottom_left, top_right, top_left])
52
53             bottom_left_values = [X_NODES[j], T_NODES[i]]
54             bottom_right_values = [X_NODES[j+1], T_NODES[i]]
55             top_right_values = [X_NODES[j+1], T_NODES[i-1]]
56             top_left_values = [X_NODES[j], T_NODES[i-1]]
57             nodal_values.append([bottom_left_values, bottom_right_values,
top_right_values])
58             nodal_values.append([bottom_left_values, top_right_values,
top_left_values])
59     return np.array(connectivity_matrix), np.array(nodal_values)
60
61 def generate_element_basis_coefficients(nodal_values):
62     '''
63     Given an element's nodal values, generates the coefficients of the basis
64     functions for that element.
65     '''
66     coefficients = []
67     for i in range(3):
68         # ex. c1 = (x1-x2)*(t1-t3) - (x1-x3)*(t1-t2)
69         ci = (nodal_values[i][0]-nodal_values[(i+1)%3][0])*(nodal_values[i][1]-
nodal_values[(i+2)%3][1]) - (nodal_values[i][0]-nodal_values[(i+2)%3][0])*(
nodal_values[i][1]-nodal_values[(i+1)%3][1])
70
71         # ex. a1 = (1/c1) * (x2t3-x3t2)
72         ai = (1/ci) * (nodal_values[(i+1)%3][0]*nodal_values[(i+2)%3][1] -
nodal_values[(i+2)%3][0]*nodal_values[(i+1)%3][1])
73
74         # ex. b1 = (1/c1) * (t3-t2)
75         bi = (1/ci) * (nodal_values[(i+2)%3][1] - nodal_values[(i+1)%3][1])
76
77         # ex. d1 = (1/c1) * (x2-x3)
78         di = (1/ci) * (nodal_values[(i+1)%3][0] - nodal_values[(i+2)%3][0])
79         coefficients.append([ai, bi, di])
80
81     return np.array(coefficients)

```

```

82
83 def generate_element_stiffness_and_force(element_nodal_values):
84     element_stiffness = np.zeros((3, 3))
85     element_force = np.zeros(3)
86
87     basis_coefficients = generate_element_basis_coefficients(
88         element_nodal_values)
89     b = basis_coefficients[:, 1]
90     d = basis_coefficients[:, 2]
91
92     for i in range(3):
93         for j in range(3):
94             # A_ij = (bj*bi + dj*di) * area
95             A_ij = (b[j]*b[i] - d[j]*d[i]) * AREA
96             element_stiffness[i, j] = A_ij
97
98     # Quadrature for the force term, using the centroid of the triangle as
99     # the quadrature point
100     # Fi = (area / 3) * f(xc, tc), where (xc, tc) is the centroid of the
101     # triangle
102     xc = np.mean(element_nodal_values[:, 0])
103     tc = np.mean(element_nodal_values[:, 1])
104     Fi = (AREA / 3) * f(xc, tc)
105     element_force[i] = Fi
106
107     return element_stiffness, element_force
108
109
110 def assemble_global_system(connectivity_matrix, nodal_values):
111     num_nodes = (N+1)*(M+1)
112     global_stiffness = np.zeros((num_nodes, num_nodes))
113     global_force = np.zeros(num_nodes)
114
115     for element_index in range(connectivity_matrix.shape[0]):
116         element_nodal_values = nodal_values[element_index]
117         element_stiffness, element_force = generate_element_stiffness_and_force
118         (element_nodal_values)
119
120         for i in range(3):
121             global_i = connectivity_matrix[element_index, i]
122             global_force[global_i] += element_force[i]
123             for j in range(3):
124                 global_j = connectivity_matrix[element_index, j]
125                 global_stiffness[global_i, global_j] += element_stiffness[i, j]
126
127     return global_stiffness, global_force
128
129
130 def fem_solver():
131     global_index_matrix = generate_global_index_matrix()

```

```

125     connectivity_matrix, nodal_values =
generate_connectivity_matrix_and_nodal_values(global_index_matrix)
126     global_stiffness, global_force = assemble_global_system(connectivity_matrix
, nodal_values)

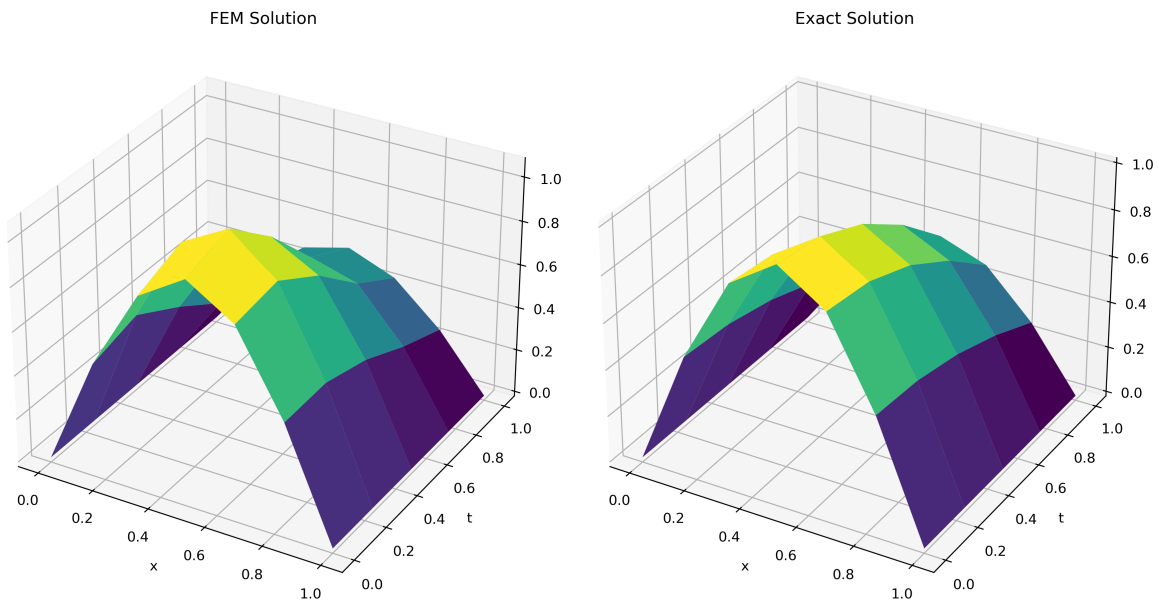
127
128     # Apply boundary conditions (u(x,0)=sin(pi*x), u(0,t)=u(1,t)=0)
129     # Initial condition (t = 0)
130     for j in range(N+1):
131         k = global_index_matrix[M, j]
132         # Zero out row
133         global_stiffness[k, :] = 0
134         # Set diagonal to 1
135         global_stiffness[k, k] = 1
136         # Set force to sin(pi*x)
137         global_force[k] = np.sin(np.pi*X_NODES[j])
138
139     # Spatial boundaries (x = 0 and x = 1)
140     for i in range(M+1):
141         # left boundary x=0
142         k = global_index_matrix[i, 0]
143         global_stiffness[k, :] = 0
144         global_stiffness[k, k] = 1
145         global_force[k] = 0
146
147         # right boundary x=1
148         k = global_index_matrix[i, N]
149         global_stiffness[k, :] = 0
150         global_stiffness[k, k] = 1
151         global_force[k] = 0
152
153     # Solve the linear system
154     solution = solve(global_stiffness, global_force)
155
156     # Reshape the solution into 2D array matching global index matrix
157     solution_grid = np.zeros((M+1, N+1))
158     for i in range(M+1):
159         for j in range(N+1):
160             k = global_index_matrix[i, j]
161             solution_grid[i, j] = solution[k]
162
163     return solution_grid
164
165 def plot_solution(solution_grid, name):
166     # plot solution in 3D
167     X = np.zeros((M+1, N+1))
168     T = np.zeros((M+1, N+1))
169     for i in range(M, -1, -1):

```

```
170         for j in range(N+1):
171             X[i, j] = X_NODES[j]
172             T[i, j] = T_NODES[M-i]
173
174     u_exact_grid = u_exact(X, T)
175     fig, ax = plt.subplots(1,2, subplot_kw={"projection": "3d"}, figsize=(12,
176 7))
177     ax[0].plot_surface(X, T, solution_grid, cmap='viridis')
178     ax[0].set_xlabel('x')
179     ax[0].set_ylabel('t')
180     ax[0].set_zlabel('u(x,t)')
181     ax[0].set_title('FEM Solution')
182
183     ax[1].plot_surface(X, T, u_exact_grid, cmap='viridis')
184     ax[1].set_xlabel('x')
185     ax[1].set_ylabel('t')
186     ax[1].set_zlabel('u(x,t)')
187     ax[1].set_title('Exact Solution')
188     plt.tight_layout()
189     plt.savefig(name ,dpi=300)
190
191 def compute_max_error(solution_grid):
192     X = np.zeros((M+1, N+1))
193     T = np.zeros((M+1, N+1))
194     for i in range(M, -1, -1):
195         for j in range(N+1):
196             X[i, j] = X_NODES[j]
197             T[i, j] = T_NODES[M-i]
198     u_exact_grid = u_exact(X, T)
199     error_grid = np.abs(solution_grid - u_exact_grid)
200     max_error = np.max(error_grid)
201     print("Max Error:", max_error)
202
203 if __name__ == "__main__":
204     solution = fem_solver()
205     plot_solution(solution, "N6_M4.png")
206     compute_max_error(solution)
```

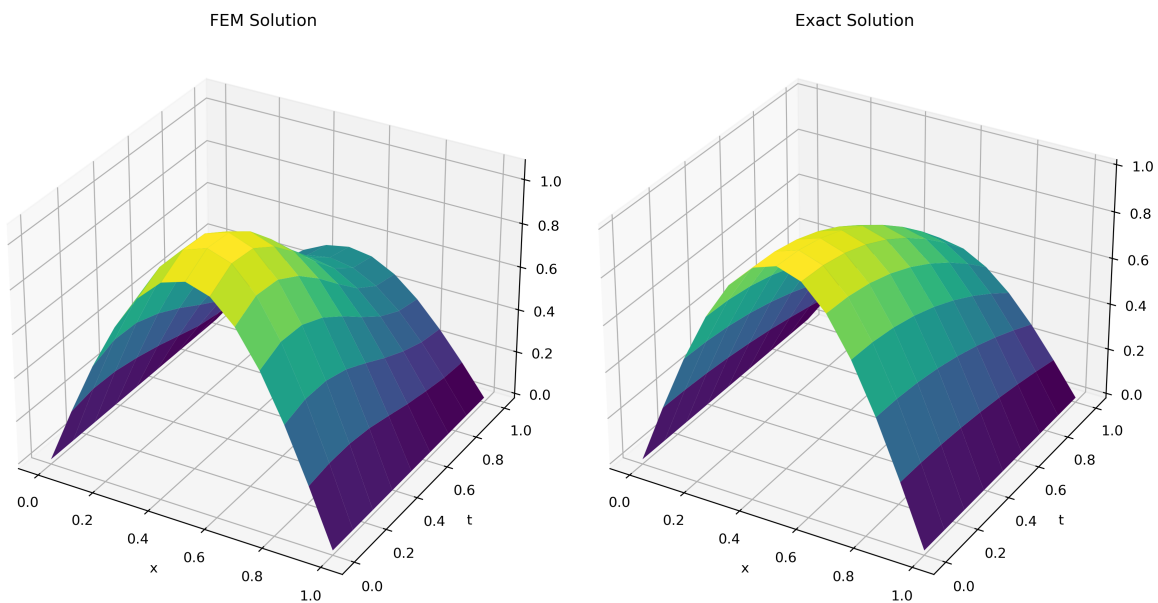
6 - Results and Bonus Questions

Given $N = 6$ and $M = 4$, we obtained the following plots:



The maximum nodal error was approximately 0.11107680006432841.

Increasing the mesh resolution to $N = 12$ and $M = 8$, we obtained the following plots:



The maximum nodal error was approximately 0.0911792636573886.

We observe that our numerical solution captures the general behavior of the exact solution. However, there are some notable discrepancies around $t = 0.8$, where the approximation appears to be dampened compared to the exact solution. This could be due to the choice of a single point quadrature rule, which may not be sufficient to capture the curvature/oscillation completely. An increase in the number of quadrature points or a higher-order quadrature method could potentially improve the accuracy in this region.

In refining the mesh, we observed a reduction in the maximum nodal error. An additional refinement of the mesh to $N = 24$ and $M = 16$ showed another decrease in error. This is consistent with the expectation that a finer mesh should yield a more accurate approximation to the exact solution.